MATH 381 Exam 1

February 6, 2026

Name (as on Gradescope): Solutions

Student #: _____

Problem:	1	2	3	Total
Points:	10	10	10	30

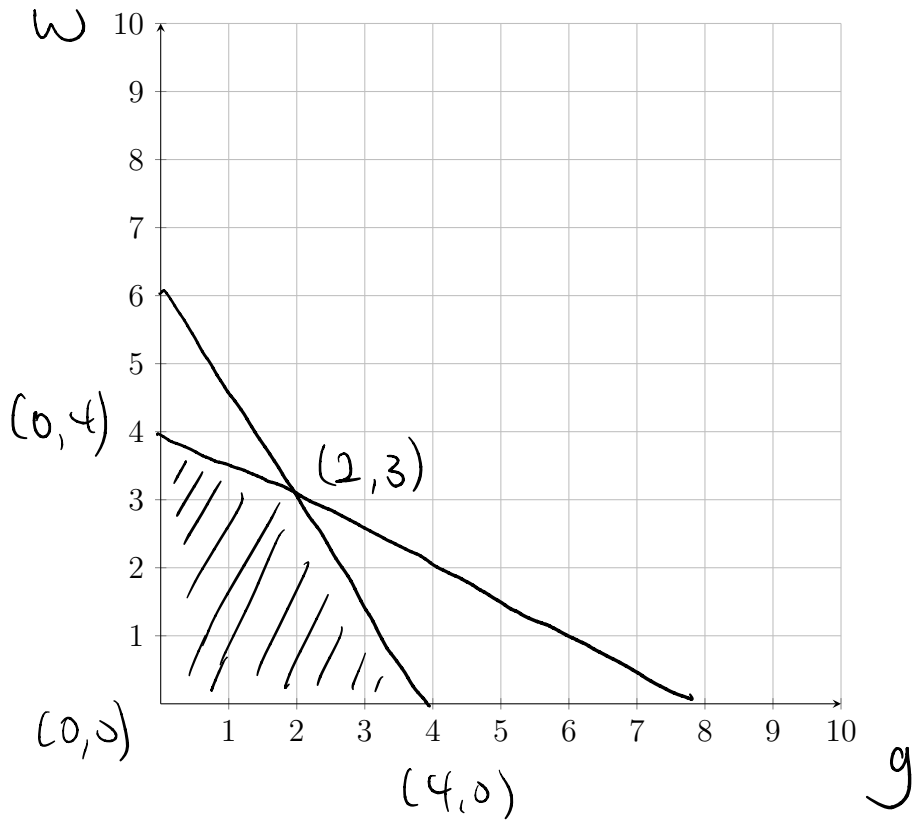
INSTRUCTIONS:

- You have 50 minutes to take the test.
- Write your solution below the problem. There is scratch paper at the back of the test.
- If you need extra space to write your solution, use the paper at the back and indicate on which page your solution continues.
- The test is double-sided. Make sure you are reading the backs of pages!
- Unless otherwise stated, show all your work for full credit.
- You are allowed to use one 8.5"×11" sheet of notes, front and back.
- Calculators are not allowed.

Good luck!

1. (10 points) A company makes two products, gadgets and widgets. To be produced, each gadget requires 3 units of aluminum and 1 unit of copper. Each widget requires 2 units of aluminum and 2 units of copper. The company has 12 units of aluminum and 8 units of copper available. Gadgets sell for a profit of \$4, and widgets sell for a profit of \$2. Assuming that all units produced are sold, how many of each product should the company make to maximize profit?

You may use the below plot. If you don't get an integral solution, you should re-check your work. Make sure to show your work for full credit (it is okay to show it on the plot).



g = # gadgets produced

w = # widgets produced

Max $4g+2w$

s.t. $3g+2w \leq 12$

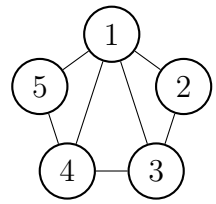
$g+2w \leq 8$

$g, w \geq 0$

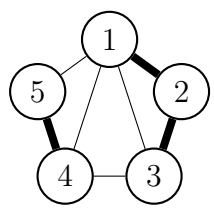
Vertices	$4g+2w$
(0,0)	0
(4,0)	16
(0,4)	8
(2,3)	14

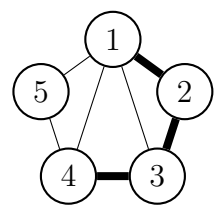
4 gadgets
0 widgets

2. (10 points) Given an undirected graph, an *edge cover* of the graph is a set of edges such that every vertex of the graph is an endpoint of at least one edge of the set. For example, consider the following graph.



The set of edges $\{12, 23, 45\}$ is an edge cover because every vertex is an endpoint of at least one of these three edges. However, $\{12, 23, 34\}$ is not an edge cover because vertex 5 is not covered. See the below figures.





Our goal is to write a function to find an edge cover of minimum size. We will use the Graph class from the homework to do this. The code for Graph is displayed on this and the next page; **do not change anything** in the Graph class. The function `min_edge_cover` on Page 5 should accept a Graph object, and output a set of edges of the graph which is a minimum-size edge cover. Complete the code on Page 5, **using the outline given**. Do not add new lines.

```
class Graph:

  def __init__(self, directed=False):
    self.vertices = set()
    self.outneighbors = {}
    self.inneighbors = {}
    self.directed = directed

  def add_vertex(self, vertex):
    if vertex in self.vertices:
      return
    self.vertices.add(vertex)
    self.outneighbors[vertex] = set()
    self.inneighbors[vertex] = set()
```

```
def add_edge(self, vertex1, vertex2):
  if not(vertex1 in self.vertices and vertex2 in self.vertices):
    return
  self.outneighbors[vertex1].add(vertex2)
  self.inneighbors[vertex2].add(vertex1)
  if not self.directed:
    self.outneighbors[vertex2].add(vertex1)
    self.inneighbors[vertex1].add(vertex2)

def graph_from_data(vertices, edges, directed=False):
  graph = Graph(directed=directed)
  for v in vertices:
    graph.add_vertex(v)
  for edge in edges:
    e = list(edge)
    if len(e) == 2:
      graph.add_edge(e[0],e[1])
    else:
      graph.add_edge(e[0],e[0])
  return graph

def get_vertices(self):
  return self.vertices

def get_inneighbors(self, vertex):
  return self.inneighbors[vertex]

def get_outneighbors(self, vertex):
  return self.outneighbors[vertex]

def get_neighbors(self, vertex):
  return self.inneighbors[vertex] | self.outneighbors[vertex]

def get_edges(self):
  if self.directed:
    return {(u,v) for u in self.vertices
                  for v in self.outneighbors[u]}
  return {frozenset({u,v}) for u in self.vertices
                           for v in self.outneighbors[u]}
```

```
from ortools.linear_solver import pywraplp

# Accepts an undirected Graph object and outputs a set of edges
# which is a minimum size edge cover.
def min_edge_cover(graph):

  # Obtains the set of vertices and the set of edges of graph.
  vertices = graph.get_vertices()
  edges = graph.get_edges()

  # Create the solver.
  solver = pywraplp.Solver.CreateSolver("SAT")

  ### Create the variables. ###
  x = {}
  for e in edges
    x[ e ] = solver.IntVar( 0 , 1 , " " )

  ### Set the objective ###
  solver.Minimize(solver.Sum( x[e] for e in edges)

  ### Create the constraints. ###
  for v in vertices
    solver.Add( solver.Sum(x[frozenset({u,v})] for u in
                                graph.get_neighbors(v)) >= 1)

  # Solve and return the solution.
  # DO NOT CHANGE ANYTHING PAST THIS POINT.
  status = solver.Solve()
  if status == pywraplp.Solver.OPTIMAL:
    return {e for e in edges if x[e].solution_value() == 1}

  # This means an error occurred.
  return None
```

3. (10 points) Consider a two-player zero-sum game with the following payoff matrix for Player 1.

		P2		
	Strategies	A	B	C
P1	A	2	−1	3
	B	−2	2	4
	C	1	2	−3

The code below finds and prints the value of the game and an optimal mixed strategy for **Player 1**. Complete the code using the outline given. Do not add lines.

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("GLOP")

### Create the variables. ###
xA = solver.NumVar(0, 1, "")
xB = solver.NumVar(0, 1, "")
xC = solver.NumVar(0, 1, "")
z  = solver.NumVar(-solver.infinity(), solver.infinity(), "")

### Set the objective. ###
solver.Maximize(z)

### Create the constraints. ###
solver.Add(z <= 2*xA - 2*xB + xC)
solver.Add(z <= -xA + 2*xB + 2*xC)
solver.Add(z <= 3*xA + 4*xB - 3*xC)
solver.Add(xA + xB + xC == 1)
# Solve and return the solution. DO NOT CHANGE ANYTHING PAST THIS POINT.
status = solver.Solve()
if status == pywraplp.Solver.OPTIMAL:
  value = solver.Objective().Value()
  strategy = ( xA.solution_value(),
               xB.solution_value(),
               xC.solution_value() )
  print(f"Value of game: {value}")
  print(f"Optimal strategy for P1: {strategy}")
else:
  print("An error occurred.")
```